Introduction

The goal of this exercise is to attempt a power calculation based for an experiment. This is based on the chosen design and linear model for the experiment, for instance a Complete Block Design, Crossover Design etc. We shall first run through an example using the Complete Block Design setup. You will then need to run a similar power calculation for your group's chosen design and setup and submit it as part of the written Protocol.

Complete Block Design Example

We now look at an example calculation for a sample Complete Blocked Design $CB[1]$ experiment in R. Note that we use fixed Block effects here as opposed to random effects.

Step 1: Linear Model and Data Generating Process

We first identify the linear model associated with our design as follows:

$$Y_i = \mu + \alpha_{j(i)} + \beta_{k(i)} + \epsilon_i$$

$$\hat{\mu} = \bar{Y}$$

$$\hat{\alpha}_{j(i)} = \bar{Y}_{j(i)} - \hat{\mu}$$

$$\hat{\beta}_{k(i)} = \bar{Y}_{k(i)} - \hat{\mu}$$

Where:

- Y_i is the response for observation i;
- μ is the overall (grand) mean;
- $\alpha_{j(i)}$ is the factor effects, where $j(i)$ indicates the factor level of the ith observation;
- $\beta_{k(i)}$ are block effects, where $k(i)$ indicates the block the ith observation belongs to;
- ϵ_i is the error term, $\epsilon_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$.

$\bar{Y}, \bar{Y}_{j(i)}, \bar{Y}_{k(i)}$ are sample means.

Write this out in R as follows:

```
# Data Generating Process outline

# Write out the linear model as an R function

generate_Y <- function(mu, alpha, beta, sigma_sq, j, k) {
  # j: n length vector of factor level indices
  # k: n length vector of block indices

  n <- length(j)

  set.seed(42)
  epsilon <- rnorm(n, mean = 0, sd = sqrt(sigma_sq))

```

```
  Y <- mu + alpha[j] + beta[k] + epsilon

  return(Y)
}
```

Step 2: Choosing Parameters
We now choose the relevant parameters for our power calculation. Set effect sizes to the smallest value that would be scientifically meaningful to you. For instance, . You may also choose these from previous literature etc. Keep in mind that your size should match conditions on these parameters (sum to zero etc)!

Next, use an estimate of the error variance σ^2, ideally from your pilot data or dry run. Set the sample size as a meaningful sample size that you feasibly hope to attain or collect while conducting your experiment.

For the purposes of our example we stick to two factor levels and two blocks, i.e 4 observations in total.

```
# Defining parameter values

mu <-
alpha <- c(,) # Fill in levels of alpha: they should sum to zero!
beta <- c(,)  # Fill in levels of beta: they should sum to zero!

sigma_sq <-
n <-
```

Note in particular the role of σ^2 in the power calculations. Once you finish steps 1-4, remember to come back and double the value of σ^2. What do you see?

Step 3: Simulating a dataset and Inference
We now simulate a dataset using the linear data-generating process and the parameters defined above. We can do this simply as follows.

Fill in the missing steps, in particular choose j and k for each observation and store it in the vectors. Sketching out a table for your experiment may be useful here!

```
# Simulating the dataset

j <- c()  # fill in factor levels
k <- c()  # fill in blocks

# Simulate response using the model
Y <- generate_Y(mu, alpha, beta, sigma_sq, j, k)

# Put into a data frame
dat <- data.frame(
  Y = Y,
  treatment = factor(j),
```

```
  block = factor(k)
)
```

We now proceed to run inference and calculate the p-value for our single factor here using the traditional ANOVA and F-test. Complete the steps!

```
# Inference on simulated data 

# Fit linear model (fixed effects for both)
fit <- lm(Y ~ # fill in linear model
          , data = dat)

# Extract ANOVA table
aov_tab <- anova(fit)

# Get p-value for treatment effect
p_val <- aov_tab["treatment", "Pr(>F)"]

# Convert to rejection indicator
reject <- as.integer(p_val < 0.05)

reject
```

Step 4: Iterating to calculate Power

Now that we have outlined the pipeline for simulating a dataset and doing inference, we compile it together into a single `R` function so that it is easy to replicate.

```
# One simulation run through of Steps 1-3

one_sim <- function(mu, alpha, beta, sigma_sq, j, k) {
  # Generate data
  Y <- generate_Y(mu, alpha, beta, sigma_sq, j, k)

  dat <- data.frame(
    Y = Y,
    treatment = factor(j),
    block = factor(k)
  )

  # Fit model
  fit <- lm(Y ~ treatment + block, data = dat)

  # Extract ANOVA p-value for treatment
  p_val <- anova(fit)["treatment", "Pr(>F)"]
```

```
  # Return rejection indicator
  return(as.integer(p_val < 0.05))
}
```

Now, we use our single-run function to iterate over many such simulated datasets under the same proposed “Alternate Hypothesis” of our chosen effect sizes. We use the Monte Carlo machinery from previous power calculations to estimate the probability of rejecting the null () under this alternate!

```
# Iterate over Steps 1-3 multiple times: Turn the Monte Carlo crank!

# Number of simulations
iter <- 1000

# Run simulations
reject_vec <- replicate(
  iter,
  one_sim(mu, alpha, beta, sigma_sq, j, k)
)

# Estimated power
power_est <- mean(reject_vec)

power_est
```

Thus, `power_est` gives us an estimate for power under our specified design and alternative (the minimum deviation from the null that we care about!). We can also run some diagnostics to make sure this estimate has correctly converged.

```
library(ggplot2)

# Running Monte Carlo estimate of power
running_power <- cumsum(reject_vec) / seq_along(reject_vec)

# Put into a data frame for ggplot
df_power <- data.frame(
  iter = seq_along(reject_vec),
  power = running_power
)

# Convergence plot
ggplot(df_power, aes(x = iter, y = power)) +
  geom_line(linewidth = 0.8) +
  geom_hline(yintercept = mean(reject_vec), linetype = "dashed") +
```

```
  labs(
    title = "Monte Carlo Power Estimate Convergence",
    x = "Number of Simulations",
    y = "Estimated Power"
  ) +
  theme_minimal()
```

If the value of the estimate starts to stay constant after a few iterations, it means that it is stable and reliable as an estimate of your power!

Feel free to go back and change your parameters and rerun the power analysis. In particular, investigate the effect of changing σ^2!

Applying this to your own Group Projects

Use the same pipeline as above, following Steps 1-4 for your own group projects!

Step 1: Update the linear model and the data generating process according to the design you choose for your experiment. Be careful about random effects, and model constraints.

Step 2: Choose the effect sizes, and sample size that best reflects your setting and the question you want to answer. Keep in mind zero-sum constraints etc!

Step 3: Fill out the design matrix or table for your experiment, and use this to fill out `i` and `j` vectors. Add other factors or terms as necessary. Simulate a dataset and hypothesis test.

Step 4: Iterate through these steps as many times as needed to ensure your power estimate has converged!

This should give you an idea of what power you can achieve with your chosen design and sample size. Discuss as a group if it is sufficient for your purposes or not. A power of 0.7 - 0.9 is usually considered ideal.

Experiment with different sample sizes to see what size is suitable for your experiment. Feel free to try out different designs, random vs fixed block effects, or play around with the number of factors, levels etc. See which of these gives you the highest power, you may use this to pin down a design/factors if you're still uncertain. Another way of increasing power is by decreasing σ^2, either by blocking, protocols, or better measurement.

Evaluate your group project ideas and submit a protocol. Best of luck with your experiments :)